pandas in python

popular library for data manipulation and analysis

==> series--single column with an index -- 1D dimension (examples:-temperature across days)

==> DataFrames--a full spread sheet pr Table--2D dimension (examples:- student table)

```
#installation
!pip install pandas
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.
```

```
#import libraries
import pandas as pd
import numpy as np
```

```
#version check
print(pd.__version__)
```

```
2.2.2
```

Series

==> series--single column with an index -- 1D dimension (examples:-temperature across days)

--> install pandas

--> import pandas

--> creation

```
#import libraries
import pandas as pd
```

```
#from list
s=pd.Series([1,2,3,4,5])
```

```
print(s)
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

```
#changing indexes
a=pd.Series([1,2,3,4,5],index=['a','b','c','d','e'])
```

```
print(a)
```

```
a    1
b    2
c    3
d    4
e    5
dtype: int64
```

```
student={
    "name":"akhil",
    "branch":"aiml"
}
se=pd.Series(student)
print(se)
```

```
name      akhil
branch     aiml
```

```
dtype: object
```

```
#arthemetic operations
b=pd.Series([1,2,3,4,5])
```

```
b*2
# b+10
# b-1
```

	0
0	2
1	4
2	6
3	8
4	10

dtype: int64

stastical operations

```
c=pd.Series([10,20,30,40,50])
```

```
c.describe()
```

	0
count	5.000000
mean	30.000000
std	15.811388
min	10.000000
25%	20.000000
50%	30.000000
75%	40.000000
max	50.000000

dtype: float64

```
c.mean()
```

```
np.float64(30.0)
```

```
c.sum()
```

```
np.int64(150)
```

```
c.median()
```

```
30.0
```

```
c.std()
```

```
15.811388300841896
```

```
c.var()
```

```
250.0
```

boolean indexing

```
d=pd.Series([10,20,30,40,50])
```

```
d[d>30]
```

	0
3	40
4	50

dtype: int64

vectorised string operations

```
names=pd.Series(['akhil','Harsha','sravan','Siva','vishnu'])
# names.str.upper()
# names.str.len()
# names.str.lower()
# names.str.startswith('A')
```

```
names.str.lower()
```

	0
0	akhil
1	harsha
2	sravan
3	siva
4	vishnu

dtype: object

```
names.str.len()
```

	0
0	5
1	6
2	6
3	4
4	6

dtype: int64

```
names.str.upper()
```

	0
0	AKHIL
1	HARSHA
2	SRAVAN
3	SIVA
4	VISHNU

dtype: object

```
names.str.startswith('A')
```

	0
0	False
1	False
2	False
3	False
4	False

dtype: bool

Dataframes in pandas

```
student={
    "name":["akhil","harsha","sravanth","siva","vishnu"],
    "branch":["aiml","aiml","aiml","aiml","aiml"]
}
```

```
s=pd.DataFrame(student)
```

```
print(s)
```

```
      name branch
0    akhil   aiml
1   harsha   aiml
2  sravanth   aiml
3     siva   aiml
4   vishnu   aiml
```

```
std=[
    {"name":"akhil","branch":"aiml"},
    {"name":"harsha","branch":"aiml"},
    {"name":"sravanth","branch":"aiml"},
]
a=pd.DataFrame(std)
```

```
print(a)
```

```
      name branch
0    akhil   aiml
1   harsha   aiml
2  sravanth   aiml
```

```
#from numpy array
arr=np.random.rand(3, 4)
c=pd.DataFrame(arr, columns=['A','B','C','D'],index=['a','b','c'])
print(c)
```

```
          A         B         C         D
a  0.667306  0.033162  0.722990  0.801540
b  0.910096  0.677047  0.984356  0.690734
c  0.954901  0.552448  0.895658  0.093627
```

```
#exploring DataFranes
data={
    'Name':['akhil','harsha','sravanth','siva','vishnu'],
    'Age':[20,21,22,23,24],
    'Score':[88,75,92,99,89],
    'Grade':['A','B','A','A','A']
}
d=pd.DataFrame(data)
```

```
# d.head()
# print(d.head(3))
# d.tail()
# print(d.tail(3))
# d.shape
# d.info
# d.describe()
# d.columns
# d.index
# d.dtypes
# d.nunique()
# d.isnull().sum()
```

```
d.head()
```

	Name	Age	Score	Grade
0	akhil	20	88	A
1	harsha	21	75	B
2	sravanth	22	92	A
3	siva	23	99	A
4	vishnu	24	89	A

```
print(d.head(3))
```

```
       Name  Age  Score Grade
0     akhil   20     88     A
1    harsha   21     75     B
2  sravanth   22     92     A
```

```
d.tail()
```

	Name	Age	Score	Grade
0	akhil	20	88	A
1	harsha	21	75	B
2	sravanth	22	92	A
3	siva	23	99	A
4	vishnu	24	89	A

```
print(d.tail(3))
```

```
       Name  Age  Score Grade
2  sravanth   22     92     A
3      siva   23     99     A
4    vishnu   24     89     A
```

```
d.shape
```

```
(5, 4)
```

```
d.info
```

```
pandas.core.frame.DataFrame.info
def info(verbose: bool | None=None, buf: WriteBuffer[str] | None=None, max_cols: int | None=None,
memory_usage: bool | str | None=None, show_counts: bool | None=None) -> None
```

```
Print a concise summary of a DataFrame.

This method prints information about a DataFrame including
the index dtype and columns, non-null values and memory usage.

Parameters
```

```
d.describe()
```

	Age	Score
count	5.000000	5.000000
mean	22.000000	88.600000
std	1.581139	8.734987
min	20.000000	75.000000
25%	21.000000	88.000000
50%	22.000000	89.000000
75%	23.000000	92.000000
max	24.000000	99.000000

selecting columns

```
# single column-> returns series
d['Name']
```

```
d.Name
```

	Name
0	akhil
1	harsha
2	sravanth
3	siva
4	vishnu

dtype: object

```
#multiple columns -> return DataFrame
d[['Name','Score']]
```

	Name	Score
0	akhil	88
1	harsha	75
2	sravanth	92
3	siva	99
4	vishnu	89

selecting rows

.loc[]--> Label Based selection

```
d.loc[0]
```

	0
Name	akhil
Age	20
Score	88
Grade	A

dtype: object

```
d.loc[0:2]
```

	Name	Age	Score	Grade
0	akhil	20	88	A
1	harsha	21	75	B
2	sravanth	22	92	A

```
d.loc[0:2,['Name','Score']]
```

	Name	Score
0	akhil	88
1	harsha	75
2	sravanth	92

.iloc[]--> position Based selection

```
d.iloc[0]
```

	0
Name	akhil
Age	20
Score	88
Grade	A

dtype: object

```
d.iloc[0:2]
```

	Name	Age	Score	Grade
0	akhil	20	88	A
1	harsha	21	75	B

```
d.iloc[0:2, 0:2]
```

	Name	Age
0	akhil	20
1	harsha	21

```
d.iloc[-1]
```

	4
Name	vishnu
Age	24
Score	89
Grade	A

dtype: object

```
#single value

d.loc[0, 'Name']
```

```
'akhil'
```

```
d.iloc[0, 0]
```

```
'akhil'
```

afternoon session

multiple commands at a time using (& , |)

```
d[(d['Score']>80) & (d['Age']<25)]
```

	Name	Age	Score	Grade
0	akhil	20	88	A
2	sravanth	22	92	A
3	siva	23	99	A
4	vishnu	24	89	A

```
d[(d['Age']<24)  |  (d['Score']>90)]
```

	Name	Age	Score	Grade
0	akhil	20	88	A
1	harsha	21	75	B
2	sravanth	22	92	A
3	siva	23	99	A

.query() --> string based filtering

```
d.query('Score>90 and Age < 25')
```

	Name	Age	Score	Grade
2	sravanth	22	92	A
3	siva	23	99	A

```
d.query('Grade=="A"')
```

	Name	Age	Score	Grade
0	akhil	20	88	A
2	sravanth	22	92	A
3	siva	23	99	A
4	vishnu	24	89	A

```
d[d['Name'].isin(['akhil'])]
```

	Name	Age	Score	Grade
0	akhil	20	88	A

```
#adding a new column
d['Example']=["a","b","c","d","e"]
```

```
print(d)
```

```
       Name  Age  Score Grade Example
0     akhil   20     88     A       a
1    harsha   21     75     B       b
2  sravanth   22     92     A       c
3      siva   23     99     A       d
4    vishnu   24     89     A       e
```

```
d['grade'] = d['Score']>=80
```

```
print(d)
```

```
       Name  Age  Score Grade Example  grade
0     akhil   20     88     A       a   True
1    harsha   21     75     B       b  False
2  sravanth   22     92     A       c   True
3      siva   23     99     A       d   True
4    vishnu   24     89     A       e   True
```

```
#modify existing column

d['Score'] = d['Score'] + 5
```

```
d['Score'] = d['Score'] - 5
```

```
print(d)
```

```
       Name  Age  Score Grade Example  grade
0     akhil   20     88     A       a   True
1    harsha   21     75     B       b  False
2  sravanth   22     92     A       c   True
3      siva   23     99     A       d   True
4    vishnu   24     89     A       e   True
```

```
#// based on external condition r function using #apply() method

def grade(Score):
  if Score > 90:
    return 'A'
  elif Score >= 80:
    return 'B'
  else:
    return "C"
d['Grade_Letter'] = d['Score'].apply(grade)
print(d)
```

```
       Name  Age  Score Grade Example  grade Grade_Letter
0     akhil   20     88     A       a   True            B
1    harsha   21     75     B       b  False            C
2  sravanth   22     92     A       c   True            A
3      siva   23     99     A       d   True            A
4    vishnu   24     89     A       e   True            B
```

```
def percentage(score):
  if(score > 90):
    return '90%'
  elif(score > 60 and score < 80):
    return '70%'
  else:
    return 'fail'

d['percentage']=d['Score'].apply(percentage)
print(d)
```

```
       Name  Age  Score Grade Example  grade Grade_Letter percentage
0     akhil   20     88     A       a   True            B       fail
1    harsha   21     75     B       b  False            C        70%
2  sravanth   22     92     A       c   True            A        90%
3      siva   23     99     A       d   True            A        90%
4    vishnu   24     89     A       e   True            B       fail
```

```
pas={
    True:"pass",
    False:'fail'
    }
pas={'a':90,'b':80,'c':70}
d['pass']=d['Example'].map(pas)

d['result']=d['grade'].map(result)
print(d)
```

```
       Name  Age  Score Grade Example  grade Grade_Letter percentage  pass  \
0     akhil   20     88     A       a   True            B       fail  90.0
1    harsha   21     75     B       b  False            C        70%  80.0
2  sravanth   22     92     A       c   True            A        90%  70.0
3      siva   23     99     A       d   True            A        90%   NaN
4    vishnu   24     89     A       e   True            B       fail   NaN

  result
0   pass
1   fail
2   pass
3   pass
4   pass
```

```
result = {
    True: "pass",
    False: "fail"
}
pas={'A':90,'B':80,'C':60}
d['result']=d['Grade'].map(pas)
d['result']=d['grade'].map(result)
print(d)
```

```
       Name  Age  Score Grade Example  grade Grade_Letter percentage  pass  \
0     akhil   20     88     A       a   True            B       fail  90.0
1    harsha   21     75     B       b  False            C        70%  80.0
2  sravanth   22     92     A       c   True            A        90%  70.0
3      siva   23     99     A       d   True            A        90%   NaN
4    vishnu   24     89     A       e   True            B       fail   NaN

  result
0   pass
1   fail
2   pass
3   pass
4   pass
```

drop

```
d.drop('percentage',axis=1,inplace=True)
```

```
print(d)
```

```
       Name  Age  Score Grade Example  grade Grade_Letter  pass result
0     akhil   20     88     A       a   True            B  90.0   pass
1    harsha   21     75     B       b  False            C  80.0   fail
2  sravanth   22     92     A       c   True            A  70.0   pass
3      siva   23     99     A       d   True            A   NaN   pass
4    vishnu   24     89     A       e   True            B   NaN   pass
```

rename

```
d.rename(columns={'Score':"Marks", 'Name':"Student"},inplace=True)
```

```
print(d)
```

```
    Student  Age  Marks Grade Example  grade Grade_Letter  pass result
0     akhil   20     88     A       a   True            B  90.0   pass
1    harsha   21     75     B       b  False            C  80.0   fail
2  sravanth   22     92     A       c   True            A  70.0   pass
3      siva   23     99     A       d   True            A   NaN   pass
4    vishnu   24     89     A       e   True            B   NaN   pass
```

```
d.isnull()
```

	Student	Age	Marks	Grade	Example	grade	Grade_Letter	pass	result
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	True	False
4	False	False	False	False	False	False	False	True	False

```
d.notnull()
```

	Student	Age	Marks	Grade	Example	grade	Grade_Letter	pass	result
0	True	True	True	True	True	True	True	True	True
1	True	True	True	True	True	True	True	True	True
2	True	True	True	True	True	True	True	True	True
3	True	True	True	True	True	True	True	False	True
4	True	True	True	True	True	True	True	False	True

```
d.isnull().sum()
```

	0
Student	0
Age	0
Marks	0
Grade	0
Example	0
grade	0
Grade_Letter	0
pass	2
result	0

dtype: int64

drop rows/columns with NaN

```
d.dropna()
```

	Student	Age	Marks	Grade	Example	grade	Grade_Letter	pass	result
0	akhil	20	88	A	a	True	B	90.0	pass
1	harsha	21	75	B	b	False	C	80.0	fail
2	sravanth	22	92	A	c	True	A	70.0	pass

```
d.dropna(thresh=7)
```

	Student	Age	Marks	Grade	Example	grade	Grade_Letter	pass	result
0	akhil	20	88	A	a	True	B	90.0	pass
1	harsha	21	75	B	b	False	C	80.0	fail
2	sravanth	22	92	A	c	True	A	70.0	pass
3	siva	23	99	A	d	True	A	NaN	pass
4	vishnu	24	89	A	e	True	B	NaN	pass

fill missing values

```
d.fillna(0)
```

	Student	Age	Marks	Grade	Example	grade	Grade_Letter	pass	result
0	akhil	20	88	A	a	True	B	90.0	pass
1	harsha	21	75	B	b	False	C	80.0	fail
2	sravanth	22	92	A	c	True	A	70.0	pass
3	siva	23	99	A	d	True	A	0.0	pass
4	vishnu	24	89	A	e	True	B	0.0	pass

```
#replace specific values
d.replace('A',np.nan, inplace=True)
```

```
print(d)
```

```
    Student  Age  Marks Grade Example  grade Grade_Letter  pass result
0     akhil   20     88   NaN       a   True            B  90.0   pass
1    harsha   21     75     B       b  False            C  80.0   fail
2  sravanth   22     92   NaN       c   True          NaN  70.0   pass
3      siva   23     99   NaN       d   True          NaN   NaN   pass
4    vishnu   24     89   NaN       e   True            B   NaN   pass
```

```
#sort by single column
d.sort_values('Marks', ascending=True)
```

	Student	Age	Marks	Grade	Example	grade	Grade_Letter	pass	result
1	harsha	21	75	B	b	False	C	80.0	fail
0	akhil	20	88	NaN	a	True	B	90.0	pass
4	vishnu	24	89	NaN	e	True	B	NaN	pass
2	sravanth	22	92	NaN	c	True	NaN	70.0	pass
3	siva	23	99	NaN	d	True	NaN	NaN	pass

```
d.sort_values('Marks', ascending=False)
```

	Student	Age	Marks	Grade	Example	grade	Grade_Letter	pass	result
1	harsha	21	75	B	b	False	C	80.0	fail

```
d.sort_values(['Age','Marks'], ascending=[True,False])
```

	Student	Age	Marks	Grade	Example	grade	Grade_Letter	pass	result
4	vishnu	24	89	NaN	e	True	B	NaN	pass
0	akhil	20	88	NaN	a	True	B	90.0	pass
3	siva	23	99	NaN	d	True	NaN	NaN	pass
1	harsha	21	75	B	b	False	C	80.0	fail
2	sravanth	22	92	NaN	c	True	NaN	70.0	pass

```
#sort by index
d.sort_index(ascending=False)
```

	Student	Age	Marks	Grade	Example	grade	Grade_Letter	pass	result
4	vishnu	24	89	NaN	e	True	B	NaN	pass
3	siva	23	99	NaN	d	True	NaN	NaN	pass
2	sravanth	22	92	NaN	c	True	NaN	70.0	pass
1	harsha	21	75	B	b	False	C	80.0	fail
0	akhil	20	88	NaN	a	True	B	90.0	pass

```
#get top n rows
d.nlargest(4,'Marks')
```

	Student	Age	Marks	Grade	Example	grade	Grade_Letter	pass	result
3	siva	23	99	NaN	d	True	NaN	NaN	pass
2	sravanth	22	92	NaN	c	True	NaN	70.0	pass
4	vishnu	24	89	NaN	e	True	B	NaN	pass
0	akhil	20	88	NaN	a	True	B	90.0	pass

```
d.nsmallest(2, 'Age')
```

	Student	Age	Marks	Grade	Example	grade	Grade_Letter	pass	result
0	akhil	20	88	NaN	a	True	B	90.0	pass
1	harsha	21	75	B	b	False	C	80.0	fail

```
df.groupby('Department')["Salary"].max()
```

```
import pandas as pd
```

```
employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4],
    'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'dept_id': [10, 20, 10, 30]
})

departments = pd.DataFrame({
```